

Relatório de CLI

- Grepper 2

A flag deste chall é a concatenação dos nomes de 3 arquivos que atendam aos seguintes requisitos em relação a seu nome ou conteúdo: o primeiro contém “Gr15” em seu nome, o segundo tem como terceiro caractere “8” e como oitavo “z”, e o último possui em seu conteúdo uma string com cinco letras “u” (podendo ser maiúsculo ou minúsculo).

1. Para pegar o nome do primeiro arquivo, utilizei o comando { `grep -rHn "Gr15" .` }, que busca a string “Gr15” de forma recursiva nos subdiretórios da pasta atual (flag r), exibe o caminho do arquivo (flag h) e mostra a linha onde o objetivo foi encontrado (flag n). Com isso, consegui o resultado: `./y7vQ9X4D7YB8qVnk/LAU8BaY3uXXDWqwg:1:MkTto6zGr15T9tfY`, o que mostra que o arquivo de nome “LAU8BaY3uXXDWqwg” é a resposta do primeiro caso e de fato possui em seu conteúdo um trecho de sua string com “Gr15”;
2. Para pegar o nome do segundo arquivo, utilizei o comando { `find . -type f | grep '/..8....z'` }, que procura em todos os arquivos no diretório e subdiretórios e filtra apenas os que tiverem em seu nome os caracteres fixos 8 na 3 posição e z na oitava posição (os pontos dentro do comando indicam caracteres quaisquer). Como resultado, consegui o caminho `./VA9QP1e4jGCv0SxK/9s8F3MJzKCfJt5gJ`, sendo o nome do segundo arquivo que buscamos “9s8F3MJzKCfJt5gJ”;
3. Para pegar o nome do terceiro arquivo, utilizei o comando { `grep -ri '[uU].*[uU].*[uU].*[uU].*[uU]' .` }, que realiza uma busca recursiva nos subdiretórios (flag r) ignorando diferenças entre maiúsculas e minúsculas (flag i), e seguindo o padrão para encontrar arquivos que contenham pelo menos 5 letras “U” (ou “u”) (ditado pelo conteúdo entre as aspas simples que contém um monte de [uU]). Como resultado, encontrei `./LCDVQxGdl202Ppgn/5B9Wp2BtAVYD4hnY:ADNuvFr0U2du2uU`, sendo “5B9Wp2BtAVYD4hnY” o nome do arquivo que queremos.

```
(gui@kalibon)-[~/Área de trabalho/GRIS/CLI/chall]
$ grep -rHn "Gr15" .
./y7vQ9X4D7YB8qVnk/LAU8BaY3uXXDWqwg:1:MkTto6zGr15T9tfY

(gui@kalibon)-[~/Área de trabalho/GRIS/CLI/chall]
$ find . -type f | grep '/..8....z'
./VA9QP1e4jGCv0SxK/9s8F3MJzKCfJt5gJ

(gui@kalibon)-[~/Área de trabalho/GRIS/CLI/chall]
$ grep -ri '[uU].*[uU].*[uU].*[uU].*[uU]' .
./LCDVQxGdl202Ppgn/5B9Wp2BtAVYD4hnY:ADNuvFr0U2du2uU
```

Assim, a flag, que é a concatenação do nome dos 3 arquivos encontrados, é igual a:

CTF-BR{LAU8BaY3uXXDWqwg9s8F3MJzKCfJt5gJ5B9Wp2BtAVYD4hnY}

- Variable-fixed

Ao invés de iniciar pelo chall “Variable”, resolvi começar pelo “Variable-fixed” para resolver os dois challs ao mesmo e também não ter o estresse de acabar encontrando uma solução unitended para o primeiro, e depois ter de pensar uma nova lógica e abordagem para o fixed.

Analisando primeiramente o código de shell script fornecido, é possível observar que a partir dos argumentos fornecidos pelo usuário, o script realiza uma série de verificações e operações com base nos valores e posições de cada coisa. No caso, a primeira delas é uma verificação que vê se o primeiro argumento do input do usuário é igual à variável “FLAG”. Caso a senha esteja incorreta, o script mostra uma mensagem de erro e encerra sua execução.

Para compreender mais sobre o funcionamento desse início do código, e se tratando de um chall sobre vulnerabilidades de shell, pensei em tentar fazer um bypass dessa primeira verificação de senha, para ver como os próximos argumentos são utilizados e ver o funcionamento geral do script. Dessa forma, para fazer o código ignorar a comparação, tentei inicialmente fazer shell injection para pular esse trecho, usando por exemplo `{ ; true }` e `{ "; echo 'Senha correta'; exit 0" }`. Minhas tentativas acabaram de um modo geral falhando, já que as respostas do servidor até podiam ter respondido ao comando injetado, mas sempre surgia o erro de “senha incorreta”, assim, resolvi trocar de forma de invasão.

Observando especificamente o trecho da comparação do primeiro argumento do input do usuário com a senha (FLAG), é possível verificar que esse input é atribuído à variável “password”, com a verificação sendo feita na linha `{ if ! [ $password = "$FLAG" ]; then }`. Nota-se que essa variável não apresenta aspas em volta (como a variável FLAG possui em sua volta), dessa maneira, passei a testar como input caracteres especiais da linguagem de shell code, que pudessem quebrar essa linha de código.

```
$ nc challs.grisufrj.com 10001
./variable.sh "; echo 'Senha correta'; exit 0"
sh: echo: unknown operand
Senha incorreta!

(gui@kalibon)~/Área de trabalho/GRIS/CLI/chall
$ nc challs.grisufrj.com 10001
./variable.sh "; true
./variable.sh $
Senha incorreta!

(gui@kalibon)~/Área de trabalho/GRIS/CLI/chall
$ nc challs.grisufrj.com 10001
./variable.sh $
Senha incorreta!

(gui@kalibon)~/Área de trabalho/GRIS/CLI/chall
$ nc challs.grisufrj.com 10001
./variable.sh "
Traceback (most recent call last):
  File "/app/./chall.py", line 9, in <module>
    args = shlex.split(user_input)
  File "/usr/local/lib/python3.13/shlex.py", line 313, in split
    return list.lex()
  File "/usr/local/lib/python3.13/shlex.py", line 300, in __next__
    token = self.get_token()
  File "/usr/local/lib/python3.13/shlex.py", line 109, in get_token
    raw = self.read_token()
  File "/usr/local/lib/python3.13/shlex.py", line 191, in read_token
    raise ValueError("No closing quotation")
ValueError: No closing quotation
```

```
(gui@kalibon)~/Área de trabalho/GRIS/CLI/chall
$ nc challs.grisufrj.com 10001
./variable.sh ""
./variable.sh: line 5: 1: 'Faltou passar a senha como argumento.'

(gui@kalibon)~/Área de trabalho/GRIS/CLI/chall
$ nc challs.grisufrj.com 10001
./variable.sh "
Traceback (most recent call last):
  File "/app/./chall.py", line 9, in <module>
    args = shlex.split(user_input)
  File "/usr/local/lib/python3.13/shlex.py", line 313, in split
    return list.lex()
  File "/usr/local/lib/python3.13/shlex.py", line 300, in __next__
    token = self.get_token()
  File "/usr/local/lib/python3.13/shlex.py", line 109, in get_token
    raw = self.read_token()
  File "/usr/local/lib/python3.13/shlex.py", line 191, in read_token
    raise ValueError("No closing quotation")
ValueError: No closing quotation
```

Casos notáveis que encontrei, por exemplo, foram ao mandar uma única aspa simples ou dupla, no qual o código realmente dava erro de execução e quando mandei duas aspas juntas, no qual o input é lido como se não houvesse nada e as aspas não fossem consideradas caracteres.

E foi aí que eu tive uma surpresa: ao testar o input { " " } (aspas duplas com um espaço entre eles), o código me deu a flag. Eu esperava que talvez esse input conseguisse bypassar a linha de código e fosse para a próxima verificação, mas o código deu erro e a descrição desse erro apresentou a flag junto, o que faz sentido já que ela aparece dentro dessa linha na comparação. O que eu percebi, com isso, é que a vulnerabilidade que acabei explorando no shell foi a interpretação incorreta dos espaços dentro de colchetes: os [] podem falhar se a variável contiver espaços não protegidos por aspas.

Após isso, testei esse input também no chall de Variable e verifiquei que de fato era a forma de resolver o chall, já que resolve ambos. Com isso, consegui as duas flags e saí feliz!

```
(gui@kalibon)-[~/Área de trabalho/GRIS/CLI/chall]
$ nc challs.grisufrrj.com 10001
./variable.sh " "
sh: CTF-BR{AGOrA_5em_uNin73nDedS_EU_3$pEr0}: unknown operand
Senha incorreta!

(gui@kalibon)-[~/Área de trabalho/GRIS/CLI/chall]
$ nc challs.grisufrrj.com 10000
./variable.sh " "
sh: CTF-BR{lE4k4ndo_Dad0s_coM_V4r!4VeIS_D0_cUR1_Lol}: unknown operand
num1 precisa ser um número.
```